

Guía rápida de Testing para el Parcial

JUnit 5 - Mockito - JaCoCo - Maven

1. Dependencias necesarias

Estas dependencias van dentro de **<dependencies>** en el archivo **pom.xml**.

```
<dependency>
<groupId>org.junit.jupiter</groupId>
<artifactId>junit-jupiter</artifactId>
<version>5.10.2</version>
<scope>test</scope>
</dependency>

<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-core</artifactId>
<version>5.11.0</version>
<scope>test</scope>
</dependency>

<dependency>
<groupId>org.mockito</groupId>
<artifactId>mockito-junit-jupiter</artifactId>
<version>5.11.0</version>
<scope>test</scope>
</dependency>
```

2. Plugin de JaCoCo

JaCoCo no va como dependencia común. Va dentro de **<build><plugins>**.

```
<plugin>
<groupId>org.jacoco</groupId>
<artifactId>jacoco-maven-plugin</artifactId>
<version>0.8.11</version>
<executions>
<execution>
<goals>
<goal>prepare-agent</goal>
</goals>
</execution>
<execution>
<id>report</id>
<phase>test</phase>
<goals>
<goal>report</goal>
</goals>
</execution>
</executions>
</plugin>
```

Usar Java 17

```
<properties>
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

Evitar Java 24 con JaCoCo 0.8.11: puede aparecer *Unsupported class file major version 68*.

3. Imports principales de JUnit

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

Asserts más usados

Sintaxis	Qué valida
<code>assertEquals(esperado, obtenido)</code>	Que ambos valores sean iguales
<code>assertTrue(condicion)</code>	Que la condición sea verdadera
<code>assertFalse(condicion)</code>	Que la condición sea falsa
<code>assertNotNull(objeto)</code>	Que el objeto no sea null
<code>assertNull(objeto)</code>	Que el objeto sea null
<code>assertThrows(Excepcion.class, () -> ...)</code>	Que se lance una excepción

Ejemplo de assertThrows

```
@Test
void noDeberiaAceptarNombreVacio() {
    assertThrows(IllegalArgumentException.class, () -> {
        new Producto("", 5);
    });
}
```

4. Anotaciones de Mockito

```
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.InjectMocks;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.mockito.Mockito.*;
```

Anotación	Uso
<code>@ExtendWith(MockitoExtension.class)</code>	Activa Mockito dentro de JUnit 5
<code>@Mock</code>	Crea una dependencia simulada
<code>@InjectMocks</code>	Crea la clase bajo prueba e inyecta los mocks
<code>@Test</code>	Marca un método como prueba

5. Plantilla mínima con Mockito

```
@ExtendWith(MockitoExtension.class)
class ProductoServicioTest {

    @Mock
    private ProductoRepositorio repositorio;

    @InjectMocks
    private ProductoServicio servicio;

    @Test
    void deberiaVenderProducto() {
        Producto producto = new Producto("Libro", 2);

        when(repositorio.buscar("Libro"))
            .thenReturn(producto);

        servicio.vender("Libro");

        assertEquals(1, producto.getStock());

        verify(repositorio).buscar("Libro");
        verify(repositorio).guardar(producto);
    }
}
```

6. Sintaxis clave de Mockito

Sintaxis	Significado
when(mock.metodo()).thenReturn(valor)	Define qué devuelve el mock
verify(mock).metodo()	Comprueba que el método fue llamado
verify(mock, times(1)).metodo()	Comprueba cantidad de llamadas
verify(mock, never()).metodo()	Comprueba que nunca se llamó
verifyNoInteractions(mock)	Comprueba cero interacciones
any()	Acepta cualquier argumento compatible

Métodos void

Un método **void**, como guardar(producto), no se configura con when(...). Se verifica con verify(...).

```
verify(repositorio).guardar(producto);

doThrow(new RuntimeException("Error"))
    .when(repositorio)
    .guardar(producto);
```

7. Orden mental para escribir un test

1	Arrange	Crear objetos y configurar mocks
2	Act	Ejecutar el método que se está probando
3	Assert	Comprobar resultado o excepción
4	Verify	Comprobar interacciones con mocks

Reglas rápidas para el parcial

- JUnit prueba comportamiento y reglas del dominio.
- Mockito simula dependencias externas como repositorios, DAOs o APIs.
- @Mock va en la dependencia; @InjectMocks va en la clase que se prueba.
- when(...).thenReturn(...) debe ir antes de ejecutar el método real.
- JaCoCo sube cobertura cuando se ejecutan líneas y ramas nuevas, no solo por agregar más tests.
- Para probar una validación posterior, los datos anteriores deben ser válidos.